

Configurable Source-Level Type Checking for Forth with Symbolic Stack Effects

Jaanus Pöial
Tallinn University of Technology

Abstract

Forth words communicate through an implicit data stack, and Forth source can also change the parser and dictionary while it is being read. This paper describes a configurable, source-level checker for that setting. A phrase is represented by an input and an output stack of typed symbolic cells. Repeated indices record that two positions contain the same abstract value. Linear phrases are checked by unifying the touching stack boundaries. At a branch, the checker must replace two path effects by one interface; it does so with a partial syntactic common-contract operation that also handles unequal visible depths when the difference is an unchanged stack prefix. Repetition is handled by comparing one execution with two, not by computing a general loop invariant.

The type hierarchy, primitive effects, parser words, defining words, literals, and structured control forms are supplied in text files. The paper gives step-by-step definitions of the three effect operators, a complete runnable example, an artifact manifest, and an exact characterization of the bottom-up computation performed after a body has been parsed. It also separates guarantees from implementation choices: normalization can discard identity information, branch combination has no accompanying semantic soundness theorem, and a concrete counterexample shows that the one-versus-two loop test does not establish correctness for arbitrary iteration counts. The native implementation is `gforth-evaluator.fs`; comparisons with StrongForth, an optional pluggable Forth type system, and WebAssembly explain which limitations arise from this checker rather than from stack languages in general.

Keywords: concatenative language; Forth; static analysis; stack effect; abstract interpretation; symbolic execution.

1 Introduction

In a concatenative language, juxtaposition is composition: if p and q are programs, then $p q$ executes p and passes its resulting stack directly to q . A conventional Forth stack annotation has the form

```
( n n -- n )
```

Such an annotation resembles a type signature, but an ordinary comment has no static force and does not generally express correlations between stack cells. For example,

```
DUP ( x[1] -- x[1] x[1] )  
OVER ( x[2] x[1] -- x[2] x[1] x[2] )
```

records facts that the unindexed signatures cannot express: two output positions may denote the same symbolic input.

This project turns such signatures into an executable abstract semantics. It loads three inputs: a finite type hierarchy, a vocabulary of word and parser specifications, and source text. It infers effects for sequences and newly defined words without executing the concrete program. The implementation is a single native Gforth source file, `gforth-evaluator.fs`.

For readers unfamiliar with Forth, a *word* is a named operation and a colon definition `: NAME ... ;` introduces a new word. Operands are normally taken from the data stack rather than from named arguments; throughout this paper the top cell is the rightmost one. Thus `1 2 +` pushes two integer literals and then replaces them by their sum. Words such as `DUP`, `SWAP`, and `OVER` respectively copy the top cell, exchange the top two cells, and copy the second cell over the top one. Control words such as `IF`, `ELSE`, and `THEN` execute while a definition is compiled and delimit source regions that run later. These data- and compile-time roles are kept separate below.

The analysis addresses four problems in addition to stack-height consistency.

1. compatibility of an effect's output boundary with the next effect's input boundary;
2. propagation of subtype refinement and symbolic identity through stack shuffles;
3. construction of a common contract for alternative branches;
4. finite approximation of unbounded iteration.

The answers implemented here are, respectively, comparable-type unification, sequence-wide substitution, a partial effect meet, and a local repeated-effect operator. These are choices made by this checker, not limitations inherent to concatenative languages. For example, WebAssembly validates loops against declared control-label types, and other typed

Forth systems make different choices about annotations and overloading (Section 12). The present choices continue the algebraic development of stack effects in [8, 9, 10, 11, 12]. An earlier Java implementation of the framework was published in 2008 [13]; the implementation studied in this paper is the native Gforth version.

The analysis distinguishes five technical properties.

- The declared type relation need not be a complete lattice. The core algorithm requires only a finite normalized relation and selects a meet when the two types are comparable.
- Raw effects and printed effects are different domains. Normalization is deterministic, but it may erase an implicitly introduced two-occurrence correlation.
- Branch merging is an operation on contracts, not the set-theoretic union of branch transition relations.
- The implemented loop operation is $\pi \sqcap (\pi \cdot \pi)$. It is not a proof that the result describes every iteration count.
- Once parsing and leaf lookup are fixed, checking is the unique bottom-up evaluation of an acyclic system of effect equations.

2 Type and Symbol Domains

2.1 Normalized finite subtype relation

Let $\mathcal{T}$ be the finite set of canonical type names. The input permits aliases; mathematically, aliases are quotiented before considering the strict subtype declarations. Write $\tau \leq v$ when τ is at least as specific as v . The bundled Forth-like profiles contain chains such as

$$a\text{-}addr < c\text{-}addr < addr < u < x, \quad char < +n < n < x.$$

The implementation adds identity, inverse lookup information, and transitive relations after loading the declarations.

Definition 1 (Comparable type meet) *For $\tau, v \in \mathcal{T}$, define the partial operation*

$$\tau \sqcap v = \begin{cases} \tau, & \tau \leq v, \\ v, & v \leq \tau, \\ \text{undefined}, & \text{otherwise.} \end{cases}$$

Thus the implementation does not search for an arbitrary third subtype when τ and v are incomparable. This is stricter than meet in a general lattice and identifies failures at the corresponding boundary.

2.2 Typed symbolic cells

A stack symbol is a triple

$$a = (\tau, i, e) \in \mathcal{T} \times \mathbb{N} \times \{0, 1\}.$$

It is printed as τ when $i = 0$ and as $\tau[i]$ otherwise. The bit e records whether an index was explicit in the specification. Symbols with the same positive index are correlated. An unindexed occurrence is freshened before composition, so two occurrences of plain $\mathbf{x}$ do not accidentally become equal.

This system uses indices as first-order equality variables, not as names of runtime storage locations. Equal indices assert one correlation; different indices do not assert that the concrete values are unequal. A substitution $[a \mapsto b]$ replaces every occurrence of a in the current effect sequence. Global replacement is what allows a type fact discovered at one composition boundary to propagate through **DUP**, **SWAP**, or **OVER**.

The brackets are an internal, explicit version of the Forth standard's numeric suffix convention. Standard prose commonly writes $\mathbf{n1} \ \mathbf{n2} \ - \ \mathbf{n3}$ to distinguish several items of type $\mathbf{n}$, while a word such as **DUP** ($\mathbf{x} \ - \ \mathbf{x} \ \mathbf{x}$) needs no suffix because every shown $\mathbf{x}$ refers to the copied item. In formulas below, $\mathbf{x}[1]$ makes the equality variable unambiguous. When effects are presented to a Forth user, indices should be renumbered consecutively and omitted only where the standard convention remains unambiguous.

3 Stack Effects and Frames

Let Σ be the set of typed symbols and let Σ^* be finite symbol sequences. The top of stack is written on the right.

Definition 2 (Raw effect) *A raw stack effect is a pair*

$$\pi = (L \ - \ - \ R), \quad L, R \in \Sigma^*.$$

It denotes a transformation of a visible stack suffix: for an arbitrary unchanged frame F , a stack of shape FL is transformed to FR , subject to the type and equality constraints represented by the symbols.

The empty effect $\mathbf{1} = (\ - \ - \)$ is identity. Algorithmic failure is represented by a separate clash value $\mathbf{0}$, not by an ordinary effect.

The implicit frame is essential. For example, $(n \ - \ -n)$ does not assert that the whole runtime stack has exactly one cell; it constrains only the suffix touched by the word. Effects with different visible depths may consequently still be combined if their depth difference is a preserved prefix. Type-system literature often calls such an arbitrary sequence a *stack row* and writes a row variable, say ρ , for it. The effect above could therefore be read $\rho n \rightarrow \rho n$. A

row variable ranges over whole stack prefixes, whereas the indexed variables of Section 2 range over single cells.

For raw effects, alpha-renaming positive indices has no meaning. Let $\pi \equiv_{\alpha} \pi'$ mean equality up to a bijective renaming of indices that preserves types and explicitness. Implementations must freshen independently instantiated word specifications before attempting unification.

4 Sequential Composition

Let

$$\pi_1 = (L_1 - -R_1), \quad \pi_2 = (L_2 - -R_2).$$

Composition inspects the touching boundary from its top inward. If the current top symbols are $a = \tau[i]$ and $b = v[j]$, calculate $\rho = \tau \sqcap v$. If undefined, composition clashes. Otherwise make a fresh symbol $z = \rho[k]$, substitute a and b by z everywhere in both working effects, remove the matched output/input pair, and continue.

A direct functional presentation follows. All indices in the two operands are assumed to be disjoint. Let $\mathcal{C}$ denote raw composition. The first two cases are

$$\mathcal{C}(L_1, \epsilon, L_2, R_2) = (L_2 L_1 - -R_2),$$

$$\mathcal{C}(L_1, R_1, \epsilon, R_2) = (L_1 - -R_1 R_2).$$

For the remaining case write $R_1 = \widehat{R}_1 a$ and $L_2 = \widehat{L}_2 b$. If the types of a and b are incomparable, $\mathcal{C}$ is undefined. Otherwise let z be their fresh, more specific representative and let $\sigma = [a \mapsto z, b \mapsto z]$. Then

$$\mathcal{C}(L_1, R_1, L_2, R_2) = \mathcal{C}(\sigma L_1, \sigma \widehat{R}_1, \sigma \widehat{L}_2, \sigma R_2).$$

This notation makes every changed sequence explicit. In the implementation, substitution mutates all four working sequences first; after that mutation the same recursive call is written $\mathcal{C}(L_1, R'_1, L'_2, R_2)$. In particular, no unexplained L'_1 or R'_2 is required. Concatenation order in the first base case reflects the implicit frame: inputs still required by π_2 lie beneath those already required by π_1 .

Define the totalized product

$$\pi_1 \odot \pi_2 = \begin{cases} \text{compact}(\pi_1 \cdot \pi_2), & \text{if raw composition succeeds,} \\ \mathbf{0}, & \text{otherwise,} \end{cases}$$

$$\mathbf{0} \odot \pi = \pi \odot \mathbf{0} = \mathbf{0}.$$

This is the operation used while reducing a source sequence.

Worked derivation. With

```

1    literal integer ( -- n )
2    literal integer ( -- n )
+      ( n n -- n )
DUP    ( x[1] -- x[1] x[1] )

```

each literal introduces a fresh numeric cell. The fragment `1 2 + DUP` then has the derivation

$$\begin{aligned}
 (- - n) \cdot (- - n) &= (- - n\ n), \\
 (- - n\ n) \cdot (n\ n - -n) &= (- - n), \\
 (- - n) \cdot (x[1] - -x[1]\ x[1]) &= (- - n[1]\ n[1]).
 \end{aligned}$$

The last step refines `x` to `n` and propagates that refinement to both outputs of `DUP`. Running this example through the native evaluator with `ex1types.txt` and `ex1specs.txt` confirms the calculation:

```

1    ( -- n )
2    ( -- n )
+    ( n n -- n[1] )
DUP  ( n[1] -- n[1] n[1] )
< n[1] n[1]

```

Proposition 1 (Determinacy of raw composition) *For fixed freshening and fresh-index policies, raw boundary composition either returns one effect or clashes. Different choices of fresh positive indices produce alpha-equivalent effects.*

Proof. At each recursive step there is one pair of top boundary symbols. The comparable meet, when it exists, is unique. Substitution and removal therefore determine the next state. The recursion decreases $|R_1| + |L_2|$ by two and terminates. Fresh-index choices can change only variable names, hence the alpha-equivalence result. $\square$

This proposition applies to raw effects. Algebraic associativity of an ideal stack-effect product motivates the polycyclic-monoid account in [8, 10]; however, inserting a lossy compaction after every binary product can affect later identity information. Consequently, algebraic laws of the raw domain do not automatically hold for the stored representation.

5 Internal Identities and Forth-Style Output

There are two separate normalization tasks. Internally, alpha-equivalent identity variables should be renamed deterministically but retained. For presentation, the Forth standard’s style numbers same-typed items only as needed and rennumbers them consecutively. For example, an internal effect

$$(x[7]\ x[8] - -x[7]\ x[9])$$

can be displayed as (`x1 x2 - x1 x3`): the first result is the first input, while the other result has no stated correlation. The special case `DUP ( x - x x )` needs no suffix because all displayed `x` items denote the same value.

The current Gforth artifact does less. After a successful operation it counts symbol occurrences over the result and its working sequence, traverses them in a deterministic order, and assigns small indices. An index remains visible if it was explicit in a source specification or if the identity occurs more than twice. An automatically introduced identity occurring once or twice is set to zero and printed without an index. Thus an implicit $(n[7] - n[7])$ becomes $(n - n)$. More seriously, the three identities in the example above can be printed as (`x x - x x`), hiding which input is preserved.

The only practical advantage of that policy is short, backward-compatible output. It gives the analysis no additional power. Standard-style numbering is preferable for inferred effects: retain every equivalence class internally, renumber classes consecutively for display, and omit suffixes only when doing so is unambiguous under the Forth convention.

Remark 1 (Current compaction is a widening) *Removing an index preserves stack shape and per-position type constraints but forgets an equality constraint. Consequently the implemented `compact` is deterministic and concise, but neither injective nor semantics-preserving for symbolic identity. It is a conservative loss of correlation information.*

This is not merely a printer issue in the current artifact: compacted effects are stored and reused. A faithful formalization must therefore define the implemented operators as “raw operation followed by compaction.” A revised implementation should keep raw identities in the dictionary and apply Forth-style formatting only at the output boundary.

6 Branches: Two Paths, One Interface

The central branch problem arises after, not inside, an `IF`. The two arms may require different visible suffixes, leave different cells, or preserve different identities, but the following word and the inferred definition need one interface. Equal stack depth is necessary but not sufficient: for example, one arm may leave an address where the other leaves a number. A checker must either retain a disjunction of path effects, forget enough facts to form a safe common summary, restrict the accepted programs, or combine these strategies. This checker chooses one partial, syntactic contract.

The operation is named `glb` in the code and is written $\sqcap$ here. The symbol $\sqcap$ does mean “meet.” It records the intended order-theoretic reading—greatest lower bound in a contract order—but the actual operator is the following algorithm, not an appeal to an unspecified lattice.

Let $\pi = (L - -R)$ and $\theta = (M - -N)$. Suppose first that $|L| > |M|$ and put $k = |L| - |M|$. A common effect is possible only if

$$|R| - |N| = k. \quad (1)$$

Write the longer sides as

$$L = P L_s, \quad R = Q R_s, \quad |P| = |Q| = k.$$

The deep prefixes P and Q are the part of the implicit frame made visible by the longer effect. The algorithm first unifies P with Q pairwise, so that this prefix is preserved. It then unifies L_s with M and R_s with N pointwise. The case $|M| > |L|$ is symmetric, and equal input lengths are the special case $k = 0$. A length failure in (1), or any incomparable pair of types, makes the operation undefined.

Here is the unequal-length case explicitly:

$$\pi = (x[1] \ n - -x[1] \ char), \quad \theta = (n - -char).$$

For π , one cell of the otherwise implicit frame is displayed. We have $k = 1$ on both sides; $x[1]$ is unchanged, and the residual $n - -char$ agrees with θ . Their common contract is therefore π . In contrast,

$$(x - -x \ x) \quad \text{and} \quad (\epsilon - -\epsilon)$$

have input-length difference one and output-length difference two, so they do not merge. This is why **IF DUP THEN**, with no corresponding **ELSE**, is rejected: only one run-time path leaves the extra cell.

With equal lengths, type refinement is pointwise. For example,

$$(x \ a\text{-}addr - -\epsilon), \quad (x \ c\text{-}addr - -\epsilon)$$

produce the stronger requirement $(x \ a\text{-}addr - -\epsilon)$. Composing the declared **IF (flag -)** effect then accounts for the selector.

The unequal-depth rule above also exposes an implementation detail. The mathematical alignment compares the residual slices at offset k . The current native helper **ev-spec-unify** starts its final vector traversal at offset zero. Consequently the artifact can reject an unequal-depth merge when the displayed frame prefix and residual suffix have different types, even though the rule above accepts it. Cases in which broad **x** cells make both alignments compatible can conceal this discrepancy. It is recorded as a current implementation limitation rather than built into the rule.

A second limitation appears even without type differences. Consider

```
: MAYSWAP  IF SWAP THEN ;
```

The true arm swaps two cells and the false arm preserves them. A conventional single Forth summary that makes no unsupported value claim is


```
( x1 x2 flag -- x3 x4 )
```

while a precise analysis would retain the disjunction “ $x2 \ x1$ or $x1 \ x2$.” The current syntactic meet instead makes the two inputs one identity and reports

```
( x[1] x[1] flag -- x[1] x[1] )
```

That restricted contract is not the general effect of `MAYSWAP`. It illustrates why a syntactic meet that strengthens preconditions must not be confused with a sound over-approximation of both run-time paths.

Thus $\sqcap$ is useful operational notation, but the name `glb` alone does not prove that the code computes a semantic greatest lower bound. Such a proof needs an operational meaning for primitive effects, an explicit variance-aware contract order, and a decision about whether path summaries may be disjunctive. No such semantic soundness theorem is claimed here.

7 Iteration: What One Versus Two Can Establish

A loop has a back edge: the abstract stack produced by one pass must be usable again at the loop header. A conventional abstract interpreter records a header state I , propagates the body through it, joins the result back into I , and iterates until a post-fixpoint is reached (with widening when needed) [4]. WebAssembly takes another route: block and loop label types are declared, and validation checks every transfer against those types. The present checker does neither. It uses a local operation inherited from the stack-effect calculus.

Definition 3 (One-versus-two repeated summary) *For an effect π , define*

$$\pi_2^* = \pi \sqcap (\pi \cdot \pi)$$

when both raw composition and the syntactic effect meet succeed; compact the result before storing it.

The test is finite and catches any nonzero net stack-depth change, because the depth changes of π and π^2 are respectively d and $2d$. Agreement requires $d = 0$. That useful depth fact must not be generalized into a claim that the returned symbolic effect is an invariant for every iteration count. The problem can be seen without subtyping.

Let `x` be the only data-cell type and declare two synthetic primitives; `NEXT?` obtains an unconstrained termination flag:

```
P      ( x[1] x[2] -- x[3] x[1] )
NEXT?  ( -- flag )
UNTIL  ( flag -- )
```

The last two effects cancel at the data boundary, so one complete loop pass has the effect of P . On concrete symbolic names, that pass maps AB to CA , two passes map it to DC , and three map it to ED . Up to consecutive renaming,

$$\begin{aligned}\pi &= (x[1] \ x[2] - -x[3] \ x[1]), \\ \pi^2 &= (x[1] \ x[2] - -x[3] \ x[4]), \\ \pi^3 &= (x[1] \ x[2] - -x[3] \ x[4]).\end{aligned}$$

Only the one-pass effect says that the top result is the original lower input. Nevertheless, the current syntactic meet can unify the unrelated variable in π^2 with that input. Because `NEXT?` may return true after any pass, this loop has arbitrary finite execution counts, yet it is accepted:

```
: TEST  BEGIN P NEXT? UNTIL ;
```

and the native artifact reports

```
TEST ( x[2] x[1] -- x[3] x[2] )
```

which is alpha-equivalent to the one-pass correlation. Composing that reported summary with itself produces the uncorrelated two-pass effect, not the reported summary. Therefore a checker requiring an inductive or idempotent summary would reject this candidate (or weaken it to an uncorrelated summary) even though the one-versus-two routine accepts it.

This example also qualifies the phrase “strictly weaker.” In a depth-only domain the one-versus-two test already proves zero net depth, and a fixpoint analysis may accept the same loop with a less precise invariant. The precise claim is not that every fixpoint checker rejects more programs. It is that $\pi \sqcap \pi^2$, with the implemented syntactic meet, is not by itself a proof that its returned type-and-identity contract describes all π^j .

The native word **ev-spec-pistar** implements exactly this local rule, and the declarative **repeat** constructor calls it. It does not calculate a control-flow fixpoint or check the result for inductiveness. The notation π_2^* is used to avoid confusing the implementation with Kleene star.

8 Declarative Source Semantics

The effect algebra is only one layer of a source checker. Forth parser words can consume names or delimited text, definitions extend the dictionary, and control words are immediate. The specification format attaches metadata such as

```
"("      parse until ")"      ( -- )
CONSTANT parse word define    ( x -- )
VARIABLE parse word define    ( -- a-addr )
:         parse word define colon ( -- )
IF        control if          ( flag -- )
```

Ordinary consumed comments are not trusted as type declarations. Thus a source comment (`- n`) cannot override inference. Recognized locals syntax may provide a provisional definition shape.

There are two different effects around a control word. For example, `IF` consumes a run-time data-stack flag, but while compiling it also places an `orig` item on the Forth control-flow stack; `THEN` resolves that item [6]. The present profile records the first as (`flag -`) and records the second only as parser metadata such as `control if` or `control fi`. Structural parsing catches misnested or missing delimiters, but the control-flow stack is not represented as a second typed stack effect. Adding effects over standard items such as `orig`, `dest`, and `do-sys` would permit compile-time stack checking independently of the run-time data-stack analysis.

Structured controls are supplied through paired `syntax:` and `effect:` descriptions. A conditional can be specified as follows (the demo spelling `FI` is a synonym for the usual `THEN`):

```
syntax:
  IF <then branch> [ELSE <else branch>] FI
effect:
  IF
  either <then branch> <else branch>
```

The effect-template language is

$$E ::= \epsilon \mid \langle s \rangle \mid c \mid E_1 E_2 \mid \text{either}(E_1, E_2) \mid \text{repeat}(E),$$

where $\langle s \rangle$ refers to a captured source segment and c is a control-word runtime effect. Its implemented semantics is

$$\begin{aligned} \llbracket \epsilon \rrbracket &= \mathbf{1}, \\ \llbracket \langle s \rangle \rrbracket &= X_s, \\ \llbracket c \rrbracket &= \Gamma(c), \\ \llbracket E_1 E_2 \rrbracket &= \llbracket E_1 \rrbracket \odot \llbracket E_2 \rrbracket, \\ \llbracket \text{either}(E_1, E_2) \rrbracket &= \overline{\cap}(\llbracket E_1 \rrbracket, \llbracket E_2 \rrbracket), \\ \llbracket \text{repeat}(E) \rrbracket &= \bar{\star}_2(\llbracket E \rrbracket), \end{aligned}$$

where bars denote totalization with $\mathbf{0}$ and include compaction after a successful raw operation.

9 The `ex1` Input/Output Example

The bundled `real` profile gives a complete, reproducible example of the evaluator interface. It consists of the three files `ex1types.txt`, `ex1specs.txt`, and `ex1prog.txt`. The launcher command is

```
./run-evaluator-gforth.sh real
```

or, equivalently, an explicit invocation of `gforth-evaluator.fs` with `-types`, `-specs`, and `-prog` arguments.

9.1 Type input

The type file declares canonical names, aliases, subtype edges, and scanner delimiters. Its essential declarations are:

```
type X cell x
type n number N
type char c
type flag boolean f
type addr a
type c-addr ca
type a-addr aa
type xt

rel n < X
rel flag < n
rel addr < X
rel char < n
rel c-addr < addr
rel a-addr < c-addr
rel xt < X
scanner EOL "\n"
```

For example, `X`, `cell`, and `x` name one canonical type, while `a-addr` is more precise than `c-addr`. The loader computes the transitive closure, so it also knows $a\text{-}addr < addr < X$.

9.2 Specification input

The specification file supplies both runtime effects and source-processing metadata. Representative entries are:

```

DUP      ( X[1] -- X[1] X[1] )
SWAP     ( X[2] X[1] -- X[1] X[2] )
"("      parse until ")" ( -- )
:        parse word define colon ( -- )
;        control end ( -- )
literal integer ( -- n )
IF       control if ( flag -- )
ELSE     control else ( -- )
FI       control fi ( -- )
BEGIN    control begin ( -- )
WHILE    control while ( flag -- )
REPEAT   control repeat ( -- )
UNTIL    control until ( flag -- )
DP       ( -- a-addr )
@        ( a-addr -- X )
C@       ( c-addr -- char )
0=       ( n -- flag )

```

Thus the same input describes ordinary words, literal recognition, parser words, defining words, and control roles. The indexed effects of `DUP` and `SWAP` preserve correlations; `@` and `C@` exercise the address subtype chain. `DP` is a system-specific demo word denoting the data-space (dictionary) pointer; it is not assumed implicitly and has the declaration `DP ( - a-addr )` shown above.

The current dictionary accepts one effect declaration per word, so it cannot directly overload `0=` with separate `( n - flag )` and `( u - flag )` entries. A profile can instead declare their common accepted supertype. For maximum Forth compatibility, the bundled `forth2012specs.txt` uses `0= ( x - flag )`; a stricter numeric profile can introduce an `integer` type above both `n` and `u` and use `( integer - flag )`. True multiple-effect selection, as supported in other Forth typing work, remains a useful extension.

9.3 Program input and inferred output

The program file defines five words and then checks one top-level sequence:

```

: PEEK2 DUP @ SWAP @ ;
: MAYSWAP IF SWAP FI ;
: NLOOP BEGIN DUP 0= WHILE REPEAT ;
: ULOOP BEGIN DUP 0= UNTIL ;
: KEEPIF IF DUP ELSE DUP FI ;

DP DP C@ 0= KEEPIF

```

The source layout is immaterial outside parsed strings and comments. The native run writes the following exact effects to `ex1prog.txt.log`:

```

PEEK2   ( a-addr[1] -- X[2] X )
MAYSWAP ( X[1] X[1] flag -- X[1] X[1] )
NLOOP   ( n[1] -- n[1] )
ULOOK   ( n[1] -- n[1] )
KEEPIF   ( X[1] flag -- X[1] X[1] )

```

The MAYSWAP line is reproducible artifact output, but it is not the proper general interface of that word. In standard-style notation the safe non-disjunctive summary is (X1 X2 flag - X3 X4); a more precise system would retain the two possible output orders. The repeated index in the actual line makes every X one symbolic value and is too restrictive, as explained in Section 6.

The top-level trace ends with

```

DP      ( -- a-addr[1] )
DP      ( -- a-addr )
C@      ( a-addr -- char )
0=      ( char -- flag )
KEEPIF   ( a-addr[1] flag -- a-addr[1] a-addr[1] )
< a-addr[1] a-addr[1]

```

Type refinement occurs at C@: the producer yields **a-addr**, while C@ requires **c-addr**. Since $a\text{-addr} < c\text{-addr}$, composition succeeds and retains the more precise address information. The flag from 0= is then consumed by KEEPIF; both branches duplicate the surviving address, yielding the final pair of correlated **a-addr** cells. The current native evaluator was run on both this complete profile and the command-line example in Section 4; both produced the outputs shown above.

9.4 Acyclic effect equations

Fix a finite parsed tree T for one source body and environment Γ . Assume every undecomposed leaf has already been assigned an effect by word lookup, literal classification, or an earlier definition. Associate a variable X_A with every tree node A :

$$\begin{aligned}
X_A &= \Gamma(A) && \text{if } A \text{ is a leaf,} \\
X_A &= X_B \odot X_C && \text{if } A \text{ sequences children } B, C, \\
X_A &= \widehat{\llbracket E \rrbracket}_X && \text{if } A \text{ applies control template } E.
\end{aligned}$$

Every right-hand side mentions only proper descendants, so the dependency graph is acyclic.

Theorem 1 (Exact characterization of parsed-body checking) *Let T and Γ satisfy the assumptions above. The induced equation system has exactly one solution in*

$$D = \{\text{compact effects}\} \cup \{\mathbf{0}\}.$$

For every segment A , the body evaluator returns its solution value X_A ; it reports a clash at A exactly when $X_A = \mathbf{0}$. The body is accepted if and only if the root value is nonzero.

Proof. Proceed by induction on node height. A leaf is fixed by Γ . At a sequence node, the induction hypothesis uniquely fixes both child values and $\odot$ is a deterministic total function on D . At a control node, structural induction on E uniquely determines the result because each constructor invokes a deterministic totalized operator. The evaluator visits the same children and applies the same operator. Therefore it computes the unique equation value at every node, including the root. $\square$

This is an exactness theorem for the implemented, post-parse computation; it is not a conventional progress-and-preservation theorem. It does not cover the outer interpreter’s evolving dictionary, provisional forward-reference summaries, recursion, or the semantic correctness of primitive specifications.

10 Native Gforth Implementation

The implementation is the single source file `gforth-evaluator.fs`. Its word families mirror the formal layers:

Formal role	Representative Gforth words
canonical types and relation	<code>ev-ts-*</code>
typed indexed symbols	<code>ev-sym-*</code>
stack-effect records	<code>ev-spec-*</code>
linear reduction and normalization	<code>ev-spec-list-*</code>
specification dictionary	<code>ev-ss-*</code>
source scanning and program parsing	<code>ev-sc-*</code> , <code>ev-parse-*</code>
diagnostics and log output	<code>ev-diag-*</code> , <code>ev-log-*</code>
command-line driver	<code>ev-parse-args</code> , <code>ev-run-native</code>

Strings, growable vectors, source spans, type symbols, effects, and control expressions are represented by explicitly allocated Gforth records. The checker normalizes CR, LF, and CRLF input, clones mutable dictionary effects before evaluation, allocates disjoint wildcard ranges, and builds the final annotation from the same normalized working value as the final effect. The launcher reserves a 4 MiB dictionary.

For effects of visible sizes m and n , boundary matching itself performs at most $\min(m, n)$ recursive matches. The actual cost is higher because each match substitutes through the current working vectors. With this representation, reducing a program of N words is roughly quadratic in the largest

intermediate symbolic effect in the worst case. A union-find representation for identities and persistent stack rows would reduce copying.

Mutation imposes three implementation obligations:

1. dictionary effects must be deep-cloned before evaluation;
2. independently evaluated branches need disjoint wildcard ranges;
3. normalization and wildcard allocation must be deterministic across repeated native runs.

All three are handled explicitly by the current Gforth source.

11 Artifact Contents and Evaluation

11.1 Manifest and reproduction

The example in Section 9 is not reproduced by the Forth-2012 files alone. Its required input trio is `ex1types.txt`, `ex1specs.txt`, and `ex1prog.txt`. The accompanying artifact contains the following paths:

Path	Role
<code>gforth-evaluator.fs</code>	native checker
<code>run-evaluator-gforth.sh</code> , <code>.bat</code>	Unix and Windows launchers
<code>ex1types.txt</code> , <code>ex1specs.txt</code> , <code>ex1prog.txt</code>	complete Section 9 profile
<code>forth2012types.txt</code> , <code>forth2012specs.txt</code> , <code>forth2012prog.txt</code>	larger Forth-2012-like profile
<code>testdata/positive</code> , <code>testdata/negative</code>	regression corpus
<code>README.md</code> , <code>EVALUATOR.md</code>	formats, commands, and limitations

From the artifact root, the exact command for the paper example is

```
./run-evaluator-gforth.sh real
```

The launcher selects the three `ex1` files and creates `ex1prog.txt.log`; that log is output, not a required input. An archive that omits the `ex1` trio cannot reproduce the named example. The essential declarations and complete source are also printed in this paper so that the example does not depend on undocumented Forth idioms.

11.2 Regression results

The checked-in corpus has 29 programs: 13 positive and 16 negative. The positive set consists of seven Forth-2012-profile examples, five custom-profile

examples, and one article profile. A fresh batch execution of the current `gforth-evaluator.fs` accepts all 13 positive programs. Fourteen of the 16 negative programs terminate with diagnostics. The two misses are `linear_clash.fs` and `top_level_clash.fs`, both involving the linear fragment DP 1+. These cases identify gaps in top-level linear checking.

Corpus class	Programs	Expected	Current Gforth result
Forth-2012 positive	7	accept	7 accepted
Custom-profile positive	5	accept	5 accepted
Article positive	1	accept	1 accepted
Negative	16	reject	14 rejected, 2 missed

The declarative Forth-2012 environment contains 50 lines of type declarations and 278 lines of vocabulary/control specifications. The current native checker is 4,424 lines of Gforth. These counts establish nontrivial scale, but they are not a performance benchmark or a soundness evaluation. In particular, the corpus is regression evidence for the implemented semantics; it cannot expose an error shared by the checker and its specifications.

12 Relation to Other Typed Stack Systems

Forth notation and algebraic effects. The Forth standard uses stack comments and numeric suffixes to distinguish same-typed items, but does not make every comment a static judgment [6, 15]. Earlier work by the present author developed algebraic composition, multiple effects, typed wild-cards, inheritance, and must-analysis operators [8, 9, 10, 11, 12]. The present artifact puts a subset of those ideas into one source-processing tool with inferred colon definitions, externally declared controls, and diagnostics. Its lack of direct multiple-effect declarations should be noted: the earlier theory is more expressive on that point than the current input format.

StrongForth. StrongForth, developed by Stephan Becher, explores a direct response to legacy compatibility: design and use a Forth system around static typing rather than require an optional checker to accept arbitrary existing Forth. Recent articles describe object-oriented typing, programming practice, and word overloading [1, 2, 3]. Overloading is particularly relevant to words such as `0=` and `EXECUTE`, for which one weak common type can lose useful distinctions. StrongForth is an integrated typed dialect and can attach language meaning to those mechanisms; the present work instead leaves the target Forth unchanged and loads an external, currently single-effect profile. The systems therefore answer different questions, and StrongForth demonstrates that the restrictions of this artifact are not restrictions of Forth-like syntax itself.

Optional, pluggable Forth typing. Riegler’s 2015 thesis is the closest comparison in goals [14]. Its Haskell prototype offers optional checkers at several levels, from stack underflow/overflow checking to stronger static type consistency, and treats subtyping, control structures, reference types, multiple stack effects, compile-time programming, object orientation, and higher-order programming. It reports that gradual checking can improve stack discipline with relatively few annotations, although higher-order uses need annotations. The current artifact contributes a native Gforth implementation, external parser/control profiles, symbolic source traces, and the parsed-body characterization of Section 9; it is narrower in multiple effects, compile-time stack typing, and higher-order features. These are complementary experiments rather than evidence that one fixed strictness works for all legacy code.

Compositional low-level typing. Stoddart and Knaggs study inference for stack-based languages [17], and Saabas and Uustalu give compositional systems for stack-based low-level code [16]. Their instruction languages are more fixed than Forth source: they do not need an open-ended outer interpreter in which a word consumes the next name or changes parsing behavior. In return, a fixed instruction and control model makes conventional typing rules and metatheory easier to state.

WebAssembly 3.0. The current online WebAssembly Core Specification identifies itself as Release 3.0, dated 23 July 2026 [18]. Wasm does not infer a loop interface by comparing one and two body executions. A block type declares the operand types consumed and produced; a control stack records each active block’s label type and stack height; and branches are checked against the target label. A loop branch targets the declared loop-label input, so every back edge is checked directly. Unreachable code receives a controlled form of stack polymorphism. The fixed bytecode and declared signatures support progress and preservation results and mechanized validation [19, 7].

The contrast locates the present trade-off. Wasm gives stronger loop and branch guarantees because its control syntax and block interfaces are fixed and explicit. This Forth checker tries to infer interfaces from annotation-light source while loading parser and control behavior as data. Symbolic identity and retargetability are additions, but compaction, syntactic branch meet, and the one-versus-two loop rule are costs of this particular design, not inherent features of concatenative type checking.

First-class concatenative functions. A *stack row* is the arbitrary untouched prefix introduced in Section 3. A row variable ρ lets one write the type of DUP schematically as “for every prefix ρ and cell type a , $\rho a \rightarrow \rho a a$.” A first-class quotation is itself a value carrying such a stack transformation.

If a word accepts a quotation that must work for *every* ρ , the universal quantifier occurs inside the argument type. This placement is what “rank- n ” (in the common case, rank-2) polymorphism means; it is more demanding than choosing ρ once for the whole call.

A recent account of the IV language avoids automatic inference of these nested quantifiers by requiring definition annotations, extending operator types across stack depths, and giving execution and composition operators explicit parameters [5]. The current checker has indexed cell variables and an implicit frame, but no type for quotation values and no algorithm that unifies explicit row variables. Supporting first-class quotations therefore requires an extension of the type language, not merely another vocabulary entry.

13 Limitations and Research Directions

The implementation realizes a restricted static analysis with the following scope limitations.

- Acceptance is relative to trusted primitive and parser specifications. An incorrect declaration can make the result unsound with respect to concrete execution.
- Comparable type meet rejects incomparable types even if the declared poset contains a useful third lower bound. The input format also has no direct multiple-effect overload for one word.
- Compaction stores away some implicit correlations instead of limiting abbreviation to Forth-style output.
- Branch meet is syntactic and may strengthen a precondition rather than compute a safe summary of both paths. **MAYSWAP** is a concrete example. In addition, the native unequal-depth helper currently aligns its residual comparison at offset zero rather than k .
- π_2^* is not an inductive loop-invariant computation; the P example in Section 7 is accepted with a non-idempotent reported summary.
- Control words are parsed structurally, but their standard compile-time control-flow-stack effects are not typed.
- Non-local effects such as **THROW**, return-stack manipulation, dynamic execution, and unrestricted metaprogramming are approximated or outside the model.
- The parsed-body theorem excludes recursive dictionary construction and forward-reference machinery.

- Two checked-in top-level negative tests are not currently rejected.

The immediate engineering priorities are to retain identity classes internally, format inferred effects in consecutive Forth-standard style, correct the unequal-depth alignment, and repair the top-level clash path. A union-find representation would avoid repeated whole-effect substitution. Language-level extensions include multiple primitive effects, an explicit compile-time control-flow stack, row variables and quotation types, disjunctive branch summaries, and a control-flow fixpoint for loops. A semantic development would add an operational model for primitives, variance-aware pre/post contracts, and a mechanized soundness argument for the parser-to-equation translation.

14 Conclusion

This work is specifically a configurable source-level checker for Forth, not a general account of all concatenative type systems. Its useful core is small: subtype-aware boundary unification composes straight-line words, indexed cells carry value correlations through stack shuffles, and external profiles describe Forth’s parser, dictionary words, and control syntax. Once a non-recursive body has been parsed, the checker performs a deterministic bottom-up evaluation of those declarations.

Branches and loops are where stack depth alone stops being enough. The paper has made the current answers explicit rather than treating the names `glb` and `pistar` as guarantees: branch combination is a partial syntactic contract with known alignment and generality problems, and repetition compares only one execution with two. The `MAYSWAP` and `P` examples delimit those claims. StrongForth, Riegler’s pluggable checker, and WebAssembly demonstrate other viable points in the design space.

The artifact is therefore useful as configurable, executable stack-effect documentation and as an experimental platform, but not yet as a sound static semantics for arbitrary Forth. Preserving identities internally, adopting standard-style output, supporting multiple effects and compile-time control items, and replacing local repetition by an inductive analysis are concrete steps from this prototype toward that stronger goal.

References

- [1] S. Becher. Statische Typprüfung und Objektorientierung. *Forth-Magazin Vierte Dimension*, 41(1):11–14, 2025. <https://forth-ev.de/wiki/res/lib/exe/fetch.php/vd-archiv:4d2025-01.pdf>.

- [2] S. Becher. Statische Typprüfung: Programmierpraxis. *Forth-Magazin Vierte Dimension*, 41(2):15–20, 2025. <https://forth-ev.de/wiki/res/lib/exe/fetch.php/vd-archiv:4d2025-02.pdf>.
- [3] S. Becher. Statische Typprüfung: Überladung von Wörtern. *Forth-Magazin Vierte Dimension*, 41(3):14–16, 2025. <https://forth-ev.de/wiki/res/lib/exe/fetch.php/vd-archiv:4d2025-03.pdf>.
- [4] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, pp. 238–252, 1977. doi:10.1145/512950.512973.
- [5] A. M. Dinikeev. Type system for a statically typed concatenate programming language with first class function support. *Information-measuring and Control Systems*, no. 5, pp. 15–25, 2025. doi:10.18127/j20700814-202505-02.
- [6] Forth 200x Standardisation Committee. *Forth 2012 Standard*. Draft proposed standard, 10 November 2014. <https://forth-standard.org/standard/intro>.
- [7] D. McDermott, Y. Morita, and T. Uustalu. A type system with subtyping for WebAssembly’s stack polymorphism. In *ICTAC 2022*, LNCS 13572, pp. 305–323, 2022. doi:10.1007/978-3-031-17715-6_20.
- [8] J. Pöial. Algebraic specifications of stack effects for Forth programs. In *1990 FORML Conference Proceedings, EuroFORML’90*, pp. 282–290, 1991.
- [9] J. Pöial. Multiple stack-effects of Forth programs. In *1991 FORML Conference Proceedings, euroFORML’91*, pp. 400–406, 1992.
- [10] J. Pöial. Algebraic specifications of stack effects. *Forth Dimensions*, 16(4):18–20, November/December 1994. <https://www.forth.org/fd/FD-V16N4.pdf>.
- [11] J. Pöial. Stack effect calculus with typed wildcards, polymorphism and inheritance. Abstract in *18th EuroForth Conference*, p. 38, 2002; accompanying presentation slides. <https://www.complang.tuwien.ac.at/anton/euroforth/ef02/poial02.pdf>.
- [12] J. Pöial. Typing tools for typeless stack languages. In *Proceedings of the 22nd EuroForth Conference*, Cambridge, England, pp. 40–46, 2006. <https://www.complang.tuwien.ac.at/anton/euroforth/ef06/poial06.pdf>.

- [13] J. Pöial. Java framework for static analysis of Forth programs. In *Proceedings of the 24th EuroForth Conference*, Vienna, Austria, pp. 20–23, 2008. <https://www.complang.tuwien.ac.at/anton/euroforth/ef08/papers/poial.pdf>.
- [14] G. Riegler. *Evaluation and Implementation of an Optional, Pluggable Type System for Forth*. Diploma thesis, Technische Universität Wien, 2015. doi:10.34726/hss.2015.27899. <https://repositum.tuwien.at/handle/20.500.12708/7117>.
- [15] E. D. Rather, D. R. Colburn, and C. H. Moore. The evolution of Forth. In *History of Programming Languages—II*, pp. 625–670, ACM, 1996. doi:10.1145/234286.1057832.
- [16] A. Saabas and T. Uustalu. Compositional type systems for stack-based low-level languages. In J. Gudmundsson and C. B. Jay, editors, *Proceedings of CATS 2006*, CRPIT 51, pp. 27–39. Australian Computer Society, 2006. <https://cs.ioc.ee/~tarmo/papers/saabas-uustalu-cats06.pdf>.
- [17] B. Stoddart and P. J. Knaggs. Type inference in stack based languages. *Formal Aspects of Computing*, 5(4):289–298, 1993. doi:10.1007/BF01212404.
- [18] WebAssembly Community Group. *WebAssembly Core Specification, Release 3.0*. Editor: A. Rossberg, 23 July 2026. <https://webassembly.github.io/spec/core/>.
- [19] C. Watt. Mechanising and verifying the WebAssembly specification. In *CPP 2018*, pp. 53–65, ACM, 2018. doi:10.1145/3167082.